

AI - ASS 1(BFS,DFS)

```
[1]: # BFS Non-Recursive
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

graph = {}

n = int(input("Enter number of nodes: "))

for i in range(n):
    node = input("Enter node: ")
    neighbors = input(f"Enter neighbors of {node} separated by space: ").split()
    graph[node] = neighbors

start = input("Enter starting node: ")

bfs(graph, start)
```

```
Enter number of nodes: 4
Enter node: A
Enter neighbors of A separated by space: B C
Enter node: B
Enter neighbors of B separated by space: D
Enter node: C
Enter neighbors of C separated by space: D
Enter node: D
Enter neighbors of D separated by space: A
Enter starting node: A
A B C D
```

```
[2]: # BFS Recursive
from collections import deque

def bfs_recursive(graph, queue, visited):
    if not queue:
        return

    node = queue.popleft()
    print(node, end=" ")

    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append(neighbor)

    bfs_recursive(graph, queue, visited)

graph = {}

n = int(input("Enter number of nodes: "))

for i in range(n):
    node = input("Enter node: ")
    neighbors = input(f"Enter neighbors of {node} separated by space: ").split()
    graph[node] = neighbors

start = input("Enter starting node: ")

visited = set([start])
queue = deque([start])

bfs_recursive(graph, queue, visited)
```

```
Enter number of nodes: 4
Enter node: A
Enter neighbors of A separated by space: B C
Enter node: B
Enter neighbors of B separated by space: D
Enter node: C
Enter neighbors of C separated by space: D
Enter node: D
Enter neighbors of D separated by space: A
Enter starting node: A
A B C D
```

```
[3]: # DFS Recursive
def dfs_recursive(graph, node, visited):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)

        for neighbor in graph[node]:
            dfs_recursive(graph, neighbor, visited)

graph = {}

n = int(input("Enter number of nodes: "))

for i in range(n):
    node = input("Enter node: ")
    neighbors = input(f"Enter neighbors of {node} separated by space: ").split()
    graph[node] = neighbors

start = input("Enter starting node: ")

visited = set()

dfs_recursive(graph, start, visited)

Enter number of nodes: 4
Enter node: A
Enter neighbors of A separated by space: B C
Enter node: B
Enter neighbors of B separated by space: D
Enter node: C
Enter neighbors of C separated by space: D
Enter node: D
Enter neighbors of D separated by space: A
Enter starting node: A
A B D C
```

```
[4]: # DFS Non-Recursive
def dfs_non_recursive(graph, start):
    visited = set()
    stack = [start]

    while stack:
        node = stack.pop()

        if node not in visited:
            print(node, end=" ")
            visited.add(node)

            for neighbor in reversed(graph[node]):
                if neighbor not in visited:
                    stack.append(neighbor)

graph = {}

n = int(input("Enter number of nodes: "))

for i in range(n):
    node = input("Enter node: ")
    neighbors = input(f"Enter neighbors of {node} separated by space: ").split()
    graph[node] = neighbors

start = input("Enter starting node: ")

dfs_non_recursive(graph, start)

Enter number of nodes: 4
Enter node: A
Enter neighbors of A separated by space: B C
Enter node: B
Enter neighbors of B separated by space: D
Enter node: C
Enter neighbors of C separated by space: D
Enter node: D
Enter neighbors of D separated by space: A
Enter starting node: A
A B D C
```